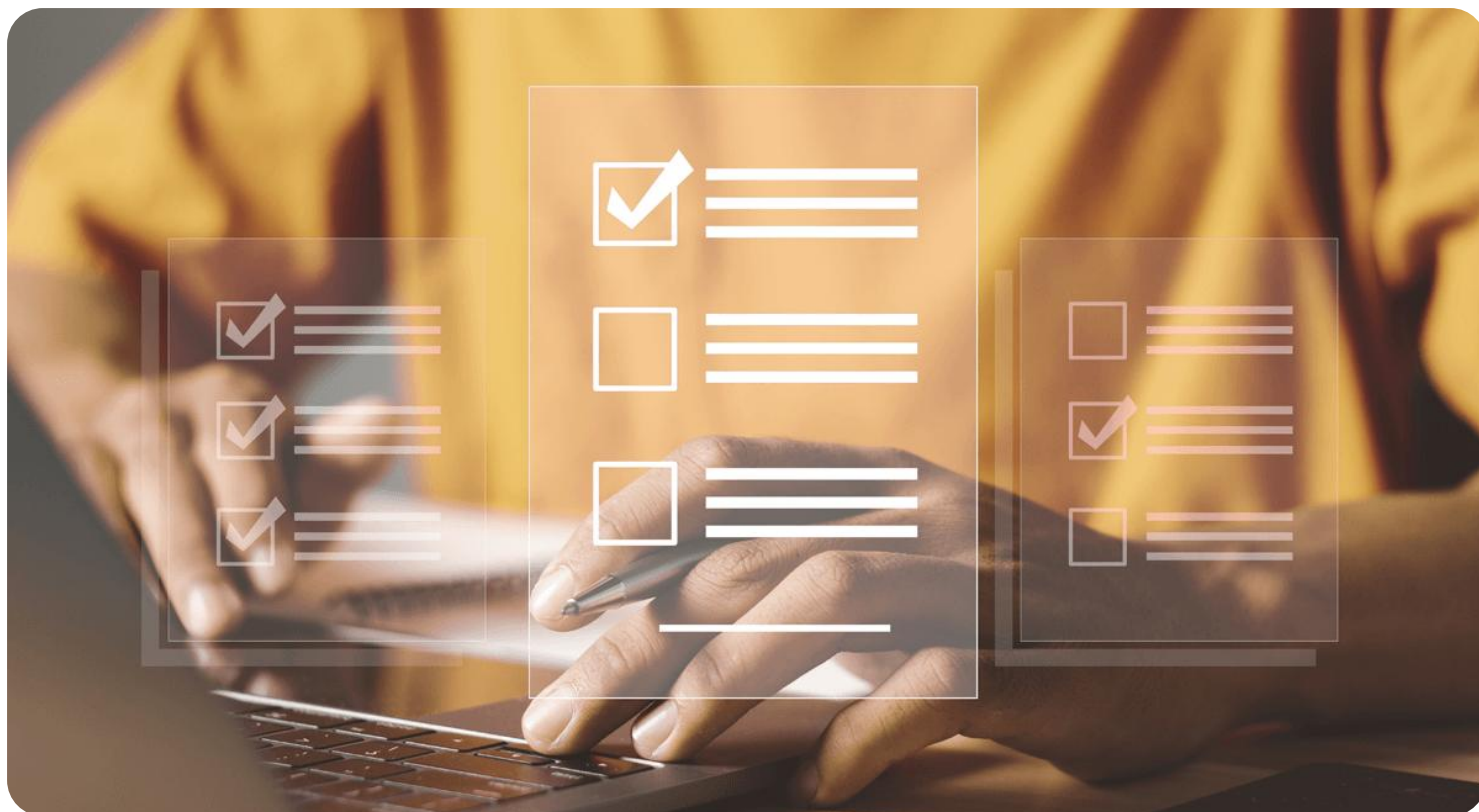




Semana 4

Desarrollo Backend II (PBY2202)

Formato de respuesta

Nombre estudiante: Raúl Zúñiga – Francisco Henríquez	
Asignatura: Desarrollo Backend II	Carrera: Desarrollo de Aplicaciones
Profesor: Jorge Carmona	Fecha: 14-06-2026

Descripción de la actividad

En esta cuarta semana, realizarás una actividad formativa grupal (2 a 3 integrantes), llamada "Configurando y Diseñando Pruebas Unitarias en Microservicios con JUnit", donde deberás configurar un entorno de pruebas unitarias en microservicios utilizando diversas herramientas de testing y frameworks, garantizando la cobertura y automatización de las pruebas. Además, tendrás que diseñar pruebas unitarias para microservicios utilizando JUnit, cumpliendo con los requerimientos especificados y estándares de la industria.

Instrucciones específicas

Para el desarrollo de la actividad formativa de la semana, tendrás que analizar el siguiente caso en base a una situación empresarial que requiere la implementación de microservicios para mejorar su arquitectura de software.

Contexto

MiniMarket Plus, una cadena en crecimiento del rubro de minimarkets en Chile, ha identificado la necesidad de optimizar la gestión de sus operaciones digitales, especialmente en relación con la gestión de ventas de productos y la administración de usuarios. En su proceso de modernización tecnológica, buscan garantizar que los microservicios relacionados con estas funcionalidades sean confiables, eficientes y cumplan con estándares de calidad establecidos.

El sistema actual permite a los usuarios registrar ventas de productos específicos, los cuales deben estar disponibles en stock y asociados a usuarios válidos. Es fundamental que estas ventas sean gestionadas adecuadamente para evitar errores como ventas sin stock, cálculos incorrectos de totales, o usuarios con datos incompletos, asegurando precisión y eficiencia operativa.

Necesidades específicas

- Gestión de Ventas:
 - Verificar que el sistema permita registrar ventas únicamente si hay stock suficiente.
 - Garantizar que el total de la venta se calcule correctamente según los precios de los productos.
- Validación de Usuarios:
 - Asegurar que cada venta esté vinculada a un usuario válido, verificando la relación entre la venta y los datos del usuario.
 - Validar que el usuario cuente con información completa (nombre, apellido, email y dirección).
- Simulación de Dependencias:
 - Utilizar herramientas de mocking para simular consultas a las entidades de usuarios y ventas, probando los microservicios de forma aislada.
- Cobertura de Código:
 - Medir y asegurar una cobertura mínima del 80% de las pruebas unitarias de las funcionalidades relacionadas con usuarios y ventas.



Importante

El código base del proyecto de "MINIMARKET PLUS" será proporcionado para su descarga a través del Aula Virtual. Deberás utilizar este backend como base para la actividad, enfocándote en la configuración del entorno de pruebas unitarias y el diseño de las pruebas necesarias para garantizar la calidad del desarrollo, según los requerimientos establecidos.

Ahora, realiza los siguientes pasos:

Paso 1: Configuración del entorno de pruebas

- Descarga y preparación: asegúrate de que el proyecto esté configurado correctamente en tu entorno de desarrollo (IDE).
- Dependencias necesarias: añade al archivo pom.xml o build.gradle las siguientes herramientas:
 - JUnit 5.
 - Mockito.
 - JaCoCo.

Paso 2: Pruebas de la entidad Usuario

- Prueba de disponibilidad:
 - Diseña una prueba que verifique que cada usuario registrado tenga sus datos obligatorios completos.
 - Usa Mockito para simular la interacción con la base de datos y garantizar que la validación de datos se realice correctamente.
- Prueba de acceso según rol:
 - Asegúrate de que solo los usuarios con roles válidos puedan ejecutar operaciones críticas como registrar ventas.

Paso 3: Pruebas de la entidad Venta

- Validación de stock de productos:
 - Diseña una prueba que verifique que los productos incluidos en una venta estén disponibles en stock suficiente.
 - Simula la verificación del stock.
- Cálculo correcto de total:
 - Asegúrate de que el total de la venta se calcule correctamente, sumando los precios de los productos.

Paso 4: Elaboración del informe de implementación

Redacta un informe técnico que integre los siguientes elementos:

1. **Resumen técnico del avance:** describe brevemente las funcionalidades implementadas previamente (semana 3) y explica qué aspectos fueron seleccionados para ser evaluados mediante pruebas unitarias esta semana.
2. **Guía de configuración:** explica los pasos realizados para integrar las herramientas necesarias al proyecto y justifica las decisiones técnicas adoptadas.
3. **Diseño y resultados de pruebas:** detalla las pruebas implementadas en las entidades Reserva y Proveedor. Incluye:
 - Explicación del enfoque utilizado para simular dependencias.
 - Detalles sobre los métodos probados y lógica de validación.
 - Evidencia de la cobertura de código alcanzada (si aplica, incluye capturas de JaCoCo).
4. **Reflexión técnica:** argumenta cómo las pruebas unitarias ayudan a mejorar la calidad del sistema y prevenir errores en producción.

Preguntas de apoyo

Para apoyarte en la reflexión de tu proyecto, guíate por las siguientes preguntas:

- ¿Qué estrategias podrías usar para simular dependencias entre ventas y usuarios en las pruebas unitarias?
- ¿Cómo validarías que las ventas sin stock no se procesen?
- ¿Qué ventajas tiene medir la cobertura de código en un sistema con múltiples entidades relacionadas?

Entregables

En este apartado deberás adjuntar lo solicitado:

1. Resumen técnico del avance

Durante la semana 03 se contó con una base funcional del minimarket basada en Spring Boot, con entidades JPA, repositorios, servicios, controladores y seguridad.

Para esta semana se seleccionaron las funcionalidades de Usuario y Venta, porque concentran la lógica de negocio más sensible del sistema:

Usuario: validación de datos obligatorios y verificación de roles permitidos.

Ventas: validación de stock y cálculo correcto del total.

Estas reglas se eligieron porque permiten prevenir errores de negocio como ventas mal calculadas, ventas sin stock o usuarios incompletos.

2. Guía de configuración del entorno de pruebas

Dependencias agregadas en pom.xml

Se incorporaron estas herramientas:

- Junit-jupiter: para realizar pruebas unitarias con JUnit 5.
- Mockito-core: para simular dependencias.
- Mockito-junit-jupiter: para integrar mockito con JUnit 5.
- Jacoco-maven-plugins: para medir la cobertura del código.

Imagen de las dependencias agregadas

```
72      <!--Para realizar las pruebas unitarias se agregó JUnit 5-->
73      <dependency>
74          <groupId>org.junit.jupiter</groupId>
75          <artifactId>junit-jupiter</artifactId>
76          <scope>test</scope>
77      </dependency>
78      <!--Se instaló mockito para simular las dependencias-->
79      <dependency>
80          <groupId>org.mockito</groupId>
81          <artifactId>mockito-core</artifactId>
82          <scope>test</scope>
83      </dependency>
84      <!--Se instaló además mockito jupiter para integrar mockito con JUnit 5-->
85      <dependency>
86          <groupId>org.mockito</groupId>
87          <artifactId>mockito-junit-jupiter</artifactId>
88          <scope>test</scope>
89      </dependency>
90  </dependencies>
```

Imagen del plugins agregado de Jacoco

```
91      <!--Aquí implementamos el plugin de Jacoco para medir la cobertura del código-->
92      <build>
93          <plugins>
94              <plugin>
95                  <groupId>org.apache.maven.plugins</groupId>
96                  <artifactId>maven-compiler-plugin</artifactId>
97                  <configuration>
98                      <annotationProcessorPaths>
99                          <path>
100                              <groupId>org.projectlombok</groupId>
101                              <artifactId>lombok</artifactId>
102                          </path>
103                      </annotationProcessorPaths>
104                  </configuration>
105              </plugin>
106              <plugin>
107                  <groupId>org.springframework.boot</groupId>
108                  <artifactId>spring-boot-maven-plugin</artifactId>
109                  <configuration>
110                      <excludes>
111                          <exclude>
112                              <groupId>org.projectlombok</groupId>
113                              <artifactId>lombok</artifactId>
114                          </exclude>
115                      </excludes>
116                  </configuration>
117              </plugin>
118              <plugin>
119                  <groupId>org.jacoco</groupId>
120                  <artifactId>jacoco-maven-plugin</artifactId>
121                  <version>0.8.12</version>
122                  <executions>
123                      <execution>
124                          <id>prepare-agent</id>
125                          <goals>
126                              <goal>prepare-agent</goal>
127                          </goals>
128                      </execution>
129                      <execution>
130                          <id>report</id>
131                          <phase>test</phase>
132                          <goals>
133                              <goal>report</goal>
134                          </goals>
135                      </execution>
136                  </executions>
137              </plugin>
138          </plugins>
139      </build>
140
141  </project>
142
```


Paso de configuración

- A- Se confirmó que el proyecto compila con java 17 y Spring Boot.
- B- Se agregaron las dependencias de prueba al pom.xml.
- C- Se configuró JaCoCo para ejecutar el agente antes de las pruebas y generar el reporte automáticamente en la fase de test
- D- Se corrigió la codificación de application.properties, para evitar errores de lectura durante Maven.
- E- Se ejecutó ./mvnw.cmd test para compilar y validar el comportamiento de las pruebas.

Justificación técnica

- **Junit 5** permite estructurar pruebas de forma clara y moderna.
- **Mockito** facilita aislar la lógica de negocio sin depender de una base de datos real.
- **JaCoCo** entrega evidencia objetiva de cobertura y permite detectar zonas de sin problemas.

3. Diseño e implementación de pruebas unitarias

Enfoque de simulación de dependencias

Se utilizó mockito para reemplazar repositorios y servicios externos:

- UsuarioRepository: fue simulado para validar búsqueda de usuarios
- VentaRepository: fue simulado para validar persistencia y consultas.
- ProductoService: fue simulado para validar stock sin consultar una base real

Este enfoque permite probar la lógica del servicio en aislamiento, que es el propósito principal de una prueba unitaria.

Pruebas implementadas para *Usuario*

Las pruebas unitarias para el proyecto que se van a realizar para los usuarios se encuentran en la siguiente ruta:

src/test/java/com/minimarket/service/imp/UsuarioServiceImplTest.java

La guía nos indica que cubramos los siguientes casos:

- Verificación de datos obligatorios completos.
- Verificación de datos incompletos.
- Validación de rol habilitado para operaciones críticas.
- Validación de rol no permitido.
- Guardado exitoso de un usuario valido.
- Rechazo de guardado cuando faltan datos.

Lógica probada:

- Se consulta el usuario con findByld.
- Se valida que nombre, apellido, email, dirección, username, password y roles no estén vacíos.
- Se revisa que el rol pertenezca al conjunto permitido: `ADMIN`, `CAJERO` o `VENDEDOR`.

Imagen del código de la prueba UsuarioServiceImplTest

```
25 //Se activa la integración entre JUnit 5 y Mockito.
26 @ExtendWith(MockitoExtension.class)
27 class UsuarioServiceImplTest { // Se declara la clase de pruebas del servicio de usuarios.
28
29     @Mock // Se crea un mock del repositorio para simular acceso a datos.
30     private UsuarioRepository usuarioRepository; // Se declara el repositorio falso que controlaremos en las pruebas.
31
32     @InjectMocks // Se inyectan los mocks en la clase real que queremos probar.
33     private UsuarioServiceImpl usuarioService; // Se crea la instancia del servicio con dependencias simuladas.
34
35     @Test // Se marca el método como una prueba unitaria.
36     void datosObligatoriosCompletos_debeRetornarTrueCuandoElUsuarioEstaCompleto() { // Se valida que un usuario con todos los datos pase la ver
37         Usuario usuario = crearUsuarioCompleto(nombreRol: "ADMIN"); // Se construye un usuario con toda la información obligatoria.
38         when(usuarioRepository.findById(id: 1L)).thenReturn(Optional.of(usuario)); // Se simula que el repositorio encuentra el usuario completo.
39         boolean resultado = usuarioService.datosObligatoriosCompletos(usuarioId: 1L); // Se ejecuta la validación real del servicio.
40         assertTrue(resultado); // Se confirma que la validación devuelve verdadero.
41         verify(usuarioRepository).findById(id: 1L); // Se confirma que el servicio consultó el repositorio.
42     } // Se cierra la prueba positiva de datos obligatorios.
43
44     @Test // Se marca el método como una prueba unitaria.
45     void datosObligatoriosCompletos_debeRetornarFalseCuandoFaltaEmail() { // Se valida que un dato faltante invalide al usuario.
46         Usuario usuario = crearUsuarioCompleto(nombreRol: "ADMIN"); // Se construye un usuario base con datos válidos.
47         usuario.setEmail(email: null); // Se elimina el correo para simular un registro incompleto.
48         when(usuarioRepository.findById(id: 2L)).thenReturn(Optional.of(usuario)); // Se simula que el usuario existe pero está incompleto.
49         boolean resultado = usuarioService.datosObligatoriosCompletos(usuarioId: 2L); // Se ejecuta la validación contra el usuario incompleto.
50         assertFalse(resultado); // Se confirma que la validación falla.
51         verify(usuarioRepository).findById(id: 2L); // Se confirma que el repositorio fue consultado.
52     } // Se cierra la prueba negativa de datos obligatorios.
53
54     @Test // Se marca el método como una prueba unitaria.
55     void tieneRolValidoParaVenta_debeRetornarTrueConRolAdmin() { // Se valida que un rol permitido habilite operaciones críticas.
56         Usuario usuario = crearUsuarioCompleto(nombreRol: "ADMIN"); // Se construye un usuario con el rol autorizado.
57         when(usuarioRepository.findById(id: 3L)).thenReturn(Optional.of(usuario)); // Se simula que el repositorio retorna el usuario con rol válido.
58         boolean resultado = usuarioService.tieneRolValidoParaVenta(usuarioId: 3L); // Se ejecuta la validación de rol.
59         assertTrue(resultado); // Se confirma que el rol permite la operación.
60         verify(usuarioRepository).findById(id: 3L); // Se verifica la consulta al repositorio.
61     } // Se cierra la prueba positiva de rol válido.
62
63     @Test // Se marca el método como una prueba unitaria.
64     void tieneRolValidoParaVenta_debeRetornarFalseConRolInvitado() { // Se valida que un rol no permitido bloquee la operación.
65         Usuario usuario = crearUsuarioCompleto(nombreRol: "INVITADO"); // Se construye un usuario con un rol fuera del conjunto permitido.
66         when(usuarioRepository.findById(id: 4L)).thenReturn(Optional.of(usuario)); // Se simula que el usuario existe en la base de datos.
67         boolean resultado = usuarioService.tieneRolValidoParaVenta(usuarioId: 4L); // Se ejecuta la validación de roles.
68         assertFalse(resultado); // Se confirma que el rol no autorizado falla la validación.
69         verify(usuarioRepository).findById(id: 4L); // Se confirma que el repositorio fue invocado.
70     } // Se cierra la prueba negativa de rol válido.
71
72     @Test // Se marca el método como una prueba unitaria.
73     void save_debePersistirUsuarioCompleto() { // Se valida que el servicio guarde usuarios completos.
74         Usuario usuario = crearUsuarioCompleto(nombreRol: "ADMIN"); // Se construye un usuario válido para persistencia.
75         when(usuarioRepository.save(usuario)).thenReturn(usuario); // Se simula que el repositorio devuelve el mismo usuario guardado.
76         Usuario guardado = usuarioService.save(usuario); // Se ejecuta la operación de guardado del servicio.
77         assertEquals(usuario, guardado); // Se confirma que el objeto devuelto es el mismo que se envió.
78         verify(usuarioRepository).save(usuario); // Se verifica que el servicio delegó la persistencia al repositorio.
79     } // Se cierra la prueba de persistencia exitosa.
80
81     @Test // Se marca el método como una prueba unitaria.
82     void save_debeLanzarExcepcionSiFaltanDatos() { // Se valida que un usuario incompleto no pueda persistirse.
83         Usuario usuario = crearUsuarioCompleto(nombreRol: "ADMIN"); // Se crea un usuario de base.
84         usuario.setDireccion(direccion: null); // Se elimina la dirección para forzar un caso inválido.
85         IllegalArgumentException exception = assertThrows(expectedType: IllegalArgumentException.class, () -> usuarioService.save(usuario)); // Se confirma que se lanza la excepción.
86         assertEquals(expected: "El usuario debe tener nombre, apellido, email, direccion, username, password y roles.", exception.getMessage()); // Se valida el mensaje de la excepción.
87         verify(usuarioRepository, never()).save(any()); // Se verifica que el repositorio no reciba la solicitud de persistencia.
88     } // Se cierra la prueba de rechazo por datos incompletos.
89
90     private Usuario crearUsuarioCompleto(String nombreRol) { // Se define un método auxiliar para construir usuarios de prueba.
91         Usuario usuario = new Usuario(); // Se crea una instancia vacía de Usuario.
92         usuario.setId(id: 1L); // Se asigna un identificador para facilitar la simulación.
93         usuario.setNombre(nombre: "Raul"); // Se asigna el nombre obligatorio.
94         usuario.setApellido(apellido: "Zuniga"); // Se asigna el apellido obligatorio.
95         usuario.setEmail(email: "raul.zuniga@minimarket.cl"); // Se asigna el correo obligatorio.
96         usuario.setDireccion(direccion: "Chile 123"); // Se asigna la dirección obligatoria.
97         usuario.setUsername(username: "raul"); // Se asigna el nombre de usuario.
98         usuario.setPassword(password: "clave123"); // Se asigna la contraseña del usuario.
99         usuario.setRoles(Set.of(new Rol(nombreRol))); // Se asigna un rol al usuario para completar la validación.
100         return usuario; // Se devuelve el usuario construido.
101     } // Se cierra el método auxiliar de creación de usuarios.
102 } // Se cierra la clase de pruebas.
103
```

Pruebas implementadas para *Venta*

Las pruebas unitarias para el proyecto que se van a realizar para las ventas se encuentran en la siguiente ruta:

src/test/java/com/minimarket/service/impl/VentaServiceImplTest.java

La guía de esta semana, nos indica que cubramos los siguientes casos:

- Validación de stock suficiente.
- Validación de stock insuficiente.
- Validación de venta nula.
- Calculo correcto del total.
- Calculo con venta nula.
- Listado completo de ventas.
- Búsqueda por id.
- Búsqueda por usuario.
- Guardado exitoso con stock suficiente.
- Rechazo de guardado cuando no hay stock.

Lógica probada:

- Se consulta el producto con `ProductoService.findById`.
- Se compara el stock real contra la cantidad solicitada.
- Se suma el total como `precio unitario x cantidad` por cada detalle.

Imagen del código de la prueba UsuarioServiceImplTest

```
29 @ExtendWith(MockitoExtension.class)
30 class VentaServiceImplTest { // Se declara la clase de pruebas del servicio de ventas.
31
32     @Mock // Se crea un mock del repositorio de ventas.
33     private VentaRepository ventaRepository; // Se declara el repositorio falso que devolverá respuestas controladas.
34
35     @Mock // Se crea un mock del servicio de productos.
36     private ProductoService productoService; // Se declara el servicio simulado que permitirá revisar stock.
37
38     @InjectMocks // Se inyectan los mocks en la clase real del servicio de ventas.
39     private VentaServiceImpl ventaService; // Se crea la instancia bajo prueba con dependencias simuladas.
40
41     @Test // Se marca el método como una prueba unitaria.
42     void tieneStockSuficiente_debeRetornarTrueCuandoElStockAlcanza() { // Se valida que el stock suficiente permita la venta.
43         Venta venta = crearVentaConDetalle(productoId: 1L, cantidad: 3, precioUnitario: 1500.0); // Se construye una venta con una cantidad menor al stock.
44         Producto producto = crearProducto(id: 1L, precio: 1500.0, stock: 10); // Se crea un producto con stock suficiente para la prueba.
45         when(productoService.findById(id: 1L)).thenReturn(producto); // Se simula la consulta al servicio de productos.
46         boolean resultado = ventaService.tieneStockSuficiente(venta); // Se ejecuta la validación de stock del servicio.
47         assertTrue(resultado); // Se confirma que la venta puede procesarse.
48         verify(productoService).findById(id: 1L); // Se verifica que se consultó el producto en la capa simulada.
49     } // Se cierra la prueba positiva de stock.
50
51     @Test // Se marca el método como una prueba unitaria.
52     void tieneStockSuficiente_debeRetornarFalseCuandoNoAlcanzaElStock() { // Se valida que una venta excedida sea rechazada.
53         Venta venta = crearVentaConDetalle(productoId: 2L, cantidad: 5, precioUnitario: 1500.0); // Se construye una venta que solicita más unidades de las disponibles.
54         Producto producto = crearProducto(id: 2L, precio: 1500.0, stock: 2); // Se crea un producto con stock insuficiente.
55         when(productoService.findById(id: 2L)).thenReturn(producto); // Se simula la consulta al servicio de productos.
56         boolean resultado = ventaService.tieneStockSuficiente(venta); // Se ejecuta la verificación de stock.
57         assertFalse(resultado); // Se confirma que la venta no cumple con el stock mínimo.
58         verify(productoService).findById(id: 2L); // Se verifica que la dependencia fue consultada.
59     } // Se cierra la prueba negativa de stock.
60
61     @Test // Se marca el método como una prueba unitaria.
62     void tieneStockSuficiente_debeRetornarFalseCuandoLaVentaEsNula() { // Se valida que una venta nula sea rechazada.
63         boolean resultado = ventaService.tieneStockSuficiente(venta: null); // Se ejecuta la validación con una referencia nula.
64         assertFalse(resultado); // Se confirma que el servicio no acepta la venta inexistente.
65         verify(productoService, never()).findById(any()); // Se confirma que no se consulta stock si la venta es nula.
66     } // Se cierra la prueba de venta nula.
67
68     @Test // Se marca el método como una prueba unitaria.
69     void calcularTotal_debeSumarCorrectamenteLosDetalles() { // Se valida que el total se calcule sumando los subtotales de cada detalle.
70         Venta venta = crearVentaConDosDetalles(); // Se construye una venta con dos productos distintos.
71         double total = ventaService.calcularTotal(venta); // Se ejecuta el cálculo de total del servicio.
72         assertEquals(expected: 6500.0, total); // Se confirma que el total coincide con la suma esperada.
73     } // Se cierra la prueba de cálculo total.
74
75     @Test // Se marca el método como una prueba unitaria.
76     void calcularTotal_debeRetornarCeroCuandoLaVentaEsNula() { // Se valida el comportamiento del cálculo cuando no hay venta.
77         double total = ventaService.calcularTotal(venta: null); // Se ejecuta el cálculo con una referencia nula.
78         assertEquals(expected: 0.0, total); // Se confirma que el total por defecto es cero.
79     } // Se cierra la prueba de total nulo.
80
81     @Test // Se marca el método como una prueba unitaria.
82     void findAll_debeRetornarTodasLasVentasRegistradas() { // Se valida que el servicio delegue la búsqueda total al repositorio.
83         Venta venta = crearVentaConDetalle(productoId: 5L, cantidad: 1, precioUnitario: 1000.0); // Se crea una venta de ejemplo para la lista simulada.
84         when(ventaRepository.findAll()).thenReturn(List.of(venta)); // Se simula el resultado del repositorio.
85         List<Venta> ventas = ventaService.findAll(); // Se ejecuta la consulta del servicio.
86         assertEquals(expected: 1, ventas.size()); // Se confirma que el servicio devuelve una sola venta.
87         assertSame(venta, ventas.get(index: 0)); // Se confirma que la venta devuelta es la misma instancia simulada.
88         verify(ventaRepository).findAll(); // Se verifica la invocación al repositorio.
89     } // Se cierra la prueba de listado general.
90
```

```

91  @Test // Se marca el método como una prueba unitaria.
92  void findById_debeRetornarVentaCuandoElIdExiste() { // Se valida que el servicio recupere una venta por identificador.
93      Venta venta = crearVentaConDetalle(productoId: 6L, cantidad: 1, precioUnitario: 1200.0); // Se construye una venta de ejemplo.
94      when(ventaRepository.findById(id: 6L)).thenReturn(java.util.Optional.of(venta)); // Se simula el hallazgo de la venta en el repositorio.
95      Venta resultado = ventaService.findById(id: 6L); // Se ejecuta la búsqueda por id.
96      assertEquals(venta, resultado); // Se confirma que la venta encontrada coincide con la simulada.
97      verify(ventaRepository).findById(id: 6L); // Se verifica que el repositorio fue consultado.
98  } // Se cierra la prueba de búsqueda por id.
99
100 @Test // Se marca el método como una prueba unitaria.
101 void findByIdUsuarioId_debeRetornarVentasDelUsuario() { // Se valida que el servicio delegue la búsqueda por usuario.
102     Venta venta = crearVentaConDetalle(productoId: 7L, cantidad: 2, precioUnitario: 1300.0); // Se crea una venta de ejemplo asociada a un usuario.
103     when(ventaRepository.findByIdUsuarioId(usuarioId: 1L)).thenReturn(List.of(venta)); // Se simula la respuesta del repositorio por usuario.
104     List<Venta> ventas = ventaService.findByIdUsuarioId(usuarioId: 1L); // Se ejecuta la búsqueda por usuario.
105     assertEquals(expected: 1, ventas.size()); // Se confirma que el servicio devuelve una venta.
106     assertEquals(venta, ventas.get(index: 0)); // Se confirma que la instancia coincide con la simulada.
107     verify(ventaRepository).findByIdUsuarioId(usuarioId: 1L); // Se verifica la interacción con el repositorio.
108 } // Se cierra la prueba de búsqueda por usuario.
109
110 @Test // Se marca el método como una prueba unitaria.
111 void save_debePersistirVentaCuandoHayStock() { // Se valida que la venta se guarde cuando hay stock suficiente.
112     Venta venta = crearVentaConDetalle(productoId: 3L, cantidad: 2, precioUnitario: 2000.0); // Se construye una venta válida para persistencia.
113     Producto producto = crearProducto(id: 3L, precio: 2000.0, stock: 8); // Se crea el producto con stock suficiente.
114     when(productoService.findById(id: 3L)).thenReturn(producto); // Se simula la búsqueda del producto en el servicio dependiente.
115     when(ventaRepository.save(venta)).thenReturn(venta); // Se simula el guardado exitoso en el repositorio.
116     Venta guardada = ventaService.save(venta); // Se ejecuta el guardado real del servicio.
117     assertEquals(venta, guardada); // Se confirma que el servicio devuelve la misma instancia persistida.
118     verify(productoService).findById(id: 3L); // Se verifica que se revisó el stock antes de guardar.
119     verify(ventaRepository).save(venta); // Se confirma que el repositorio recibió la orden de persistencia.
120 } // Se cierra la prueba de persistencia exitosa.
121

```

```

122 @Test // Se marca el método como una prueba unitaria.
123 void save_debeLanzarExcepcionCuandoNoHayStock() { // Se valida que la venta no se guarde si el stock es insuficiente.
124     Venta venta = crearVentaConDetalle(productoId: 4L, cantidad: 9, precioUnitario: 1200.0); // Se construye una venta que excede el stock.
125     Producto producto = crearProducto(id: 4L, precio: 1200.0, stock: 1); // Se crea un producto con stock menor al solicitado.
126     when(productoService.findById(id: 4L)).thenReturn(producto); // Se simula la consulta del producto en la capa de datos.
127     IllegalArgumentException exception = assertThrows(expectedType: IllegalArgumentException.class, () -> ventaService.save(venta)); // Se confirma la excepción.
128     assertEquals(expected: "No es posible registrar la venta porque uno o más productos no tienen stock suficiente.", exception.getMessage()); // Se verifica el mensaje de la excepción.
129     verify(productoService).findById(id: 4L); // Se verifica que el stock fue revisado.
130     verify(ventaRepository, never()).save(any()); // Se confirma que el repositorio no recibió la venta inválida.
131 } // Se cierra la prueba de rechazo por stock insuficiente.
132
133 private Producto crearProducto(Long id, Double precio, Integer stock) { // Se define un método auxiliar para construir productos de prueba.
134     Producto producto = new Producto(); // Se crea una instancia vacía de Producto.
135     producto.setId(id); // Se asigna el identificador del producto.
136     producto.setNombre("Producto " + id); // Se asigna un nombre de ejemplo.
137     producto.setPrecio(precio); // Se asigna el precio del producto.
138     producto.setStock(stock); // Se asigna el stock disponible.
139     producto.setCategoria(crearCategoria()); // Se asigna una categoría válida para completar la entidad.
140     return producto; // Se devuelve el producto construido.
141 } // Se cierra el método auxiliar de producto.
142
143 private Categoria crearCategoria() { // Se define un método auxiliar para construir la categoría requerida por el producto.
144     Categoria categoria = new Categoria(); // Se crea una instancia vacía de Categoría.
145     categoria.setId(id: 1L); // Se asigna un identificador de categoría.
146     categoria.setNombre(nombre: "Abarrotes"); // Se asigna un nombre de categoría de ejemplo.
147     return categoria; // Se devuelve la categoría construida.
148 } // Se cierra el método auxiliar de categoría.
149

```

```

150 private Venta crearVentaConDetalle(Long productoId, Integer cantidad, Double precioUnitario) { // Se define un método auxiliar para construir un
151     Venta venta = new Venta(); // Se crea una instancia vacía de Venta.
152     venta.setId(id: 1L); // Se asigna un identificador para la venta de prueba.
153     venta.setUsuario(crearUsuario()); // Se asocia un usuario válido a la venta.
154     venta.setDetalles(List.of(crearDetalle(productoId, cantidad, precioUnitario))); // Se agregan los detalles que serán evaluados por el servicio
155     return venta; // Se devuelve la venta construida.
156 } // Se cierra el método auxiliar de venta simple.
157
158 private Venta crearVentaConDosDetalles() { // Se define un método auxiliar para construir una venta con dos líneas de detalle.
159     Venta venta = new Venta(); // Se crea una instancia vacía de Venta.
160     venta.setId(id: 2L); // Se asigna un identificador distinto para la segunda venta.
161     venta.setUsuario(crearUsuario()); // Se asocia el mismo usuario de prueba.
162     venta.setDetalles(List.of( // Se construye la lista de detalles de la venta.
163         crearDetalle(productoId: 10L, cantidad: 2, precioUnitario: 1500.0), // Se agrega el primer detalle con subtotal 3000.
164         crearDetalle(productoId: 11L, cantidad: 1, precioUnitario: 3500.0) // Se agrega el segundo detalle con subtotal 3500.
165     )); // Se cierra la lista de detalles.
166     return venta; // Se devuelve la venta construida.
167 } // Se cierra el método auxiliar de venta múltiple.
168
169 private DetalleVenta crearDetalle(Long productoId, Integer cantidad, Double precioUnitario) { // Se define un método auxiliar para construir un
170     DetalleVenta detalle = new DetalleVenta(); // Se crea una instancia vacía de DetalleVenta.
171     detalle.setId(productoId); // Se asigna un id técnico al detalle para facilitar el seguimiento.
172     detalle.setProducto(crearProducto(productoId, precioUnitario, stock: 99)); // Se asocia un producto con stock alto para no interferir con el
173     detalle.getProducto().setId(productoId); // Se refuerza el id del producto para que coincida con la búsqueda simulada.
174     detalle.setCantidad(cantidad); // Se asigna la cantidad vendida.
175     detalle.setPrecio(precioUnitario); // Se asigna el precio unitario usado por el cálculo del total.
176     return detalle; // Se devuelve el detalle construido.
177 } // Se cierra el método auxiliar de detalle.
178
179 private Usuario crearUsuario() { // Se define un método auxiliar para construir el usuario asociado a la venta.
180     Usuario usuario = new Usuario(); // Se crea una instancia vacía de Usuario.
181     usuario.setId(id: 1L); // Se asigna un identificador de prueba.
182     usuario.setNombre(nombre: "Raul"); // Se asigna el nombre del usuario.
183     usuario.setApellido(apellido: "Zuniga"); // Se asigna el apellido del usuario.
184     usuario.setEmail(email: "raul.zuniga@minimarket.cl"); // Se asigna el correo del usuario.
185     usuario.setDireccion(direccion: "Chile 123"); // Se asigna la dirección del usuario.
186     usuario.setUsername(username: "raul"); // Se asigna el nombre de usuario.
187     usuario.setPassword(password: "clave123"); // Se asigna la contraseña del usuario.
188     return usuario; // Se devuelve el usuario construido.
189 } // Se cierra el método auxiliar del usuario.
190 } // Se cierra la clase de pruebas.
191

```

Evidencia de cobertura

La utilización de JaCoCo (Java Code Coverage) la revisamos en el ramo de Seguridad y Calidad en el desarrollo. JaCoCo es una herramienta que se utiliza para medir la cobertura de código durante la ejecución de pruebas unitarias e integradas en aplicaciones Java.

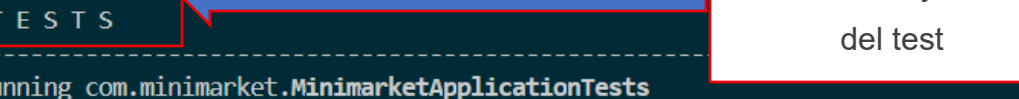
Para ejecutar JaCoCo, debemos escribir en la terminal del proyecto `./mvnw.cmd test`. De esta forma JaCoCo se comenzará a ejecutar las pruebas unitarias completas.

Imagen de la ejecución de las pruebas.

Comando para iniciar
la ejecución de las
pruebas unitarias

```
C:\Users\rzuni\iCloudDrive\Raul\DUOC\DUOC - Desarrollo de Aplicaciones
ro servicios con JUnit\minimarket> ./mvnw.cmd test
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.minimarket:minimarket >-----
[INFO] Building minimarket 0.0.1-SNAPSHOT
[INFO]   from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- jacoco:0.8.12:prepare-agent (prepare-agent) @ minimarket ---
[INFO] argLine set to "-javaagent:C:\\Users\\rzuni\\.m2\\repository\\org\\jacoco\\org.jacoco.agent
rive\\Raul\\DUOC\\DUOC - Desarrollo de Aplicaciones_\\07 - Bimestre\\02 - Desarrollo Backend II\\S
nit\\minimarket\\target\\jacoco.exec"
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ minimarket ---
[INFO] Copying 1 resource from src\\main\\resources to target\\classes
[INFO] Copying 0 resource from src\\main\\resources to target\\classes
[INFO]
[INFO] --- compiler:3.13.0:compile (default-compile) @ minimarket ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
```

Inicio de la ejecución
del test



Inicio de la ejecución del test

```
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.minimarket.MinimarketApplicationTests
12:29:56.956 [main] INFO org.springframework.test.context.support.AnnotationConfigContextLoa
ket.MinimarketApplicationTests]: MinimarketApplicationTests does not declare any static, nor
12:29:57.233 [main] INFO org.springframework.boot.test.context.SpringBootTestContextBootstra
lass com.minimarket.MinimarketApplicationTests
12:29:57.556 [main] INFO org.springframework.boot.devtools.restart.RestartApplicationListene
```

Se realizaron 20 pruebas, de las cuales no se presentaron fallas, errores o saltos

```
[INFO] Tests run: 20, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jacoco:0.8.12:report (report) @ minimarket ---
[INFO] Loading execution data file C:\Users\rzuni\ICloud\Documents\workspace\minimarket\minimarket-jar\build\reports\jacoco\jacoco.exec
[INFO] Señalando pruebas unitarias en micro servicios con JUnit\minimarket-jar\build\reports\jacoco\jacoco.exec
[INFO] Analyzed bundle 'minimarket' with 30 classes
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 13.930 s
[INFO] Finished at: 2026-06-14T12:30:07-04:00
[INFO] -----
```

Se realizaron 20 pruebas, de las cuales no se presentaron fallas, errores o saltos

Los resultados que nos entrega JaCoCo quedan registrados en la siguiente ruta:

target\site\jacoco\index.html

Resultado obtenido con JaCoCo:

minimarket

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
com.minimarket.controller		6 %		0 %	53 61	90 98	35 43	0 8
com.minimarket.service.impl		63 %		62 %	55 89	37 101	29 52	0 8
com.minimarket.entity		53 %		n/a	38 83	57 126	38 83	0 8
com.minimarket.security.model		0 %		n/a	10 10	13 13	10 10	2 2
com.minimarket.security.service		13 %		n/a	2 3	3 4	2 3	0 1
com.minimarket		37 %		n/a	1 2	2 3	1 2	0 1
com.minimarket.security.util		0 %		n/a	1 1	1 1	1 1	1 1
com.minimarket.security.config		100 %		n/a	0 8	0 17	0 8	0 1
Total	760 of 1.314	42 %	64 of 110	41 %	160 257	203 363	116 202	3 30

La interpretación del informe es el siguiente:

Métricas	Resultados
Cobertura de instrucciones	42%
Cobertura de ramas (branches)	41%
Instrucciones cubiertas	554 de 1314
Instrucciones no cubiertas	760 de 1314
Ramas cubiertas	46 de 110
Ramas no cubiertas	64 de 110
Líneas cubiertas	160 de 363
Métodos cubiertos	86 de 202
Clases cubiertas	27 de 30

Por lo tanto:

- El 42% de cobertura significa que las pruebas que realizamos están ejecutando menos de la mitad del código, siendo esto obvio ya que solamente estamos probando una parte de los módulos de todo el código.
- En la mayoría de los proyectos académicos se suele exigir al menos entre el 70% y 80% como mínimo.
- El proyecto tiene bastantes clases cubiertas (27/30), pero muchas líneas y métodos no están siendo ejecutado por el test.

Pruebas unitarias solo a un método

Si quisiéramos analizar las pruebas unitarias realizadas a un solo método, se puede hacer utilizando el siguiente comando `./mvnw.cmd -Dtest=UsuarioServiceImplTest test`. Para este caso se realizó la prueba para `UsuarioServiceImplTest`.

Imagen de la aplicación del comando

```
PS C:\Users\rzuni\iCloudDrive\Raul\DUOC\DUOC - Desarrollo de Aplicaciones_\07 - Bimestre 02 - Desarrollo de servicios con JUnit\minimarket> ./mvnw.cmd -Dtest=UsuarioServiceImplTest test
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.minimarket:minimarket >-----
[INFO] Building minimarket 0.0.1-SNAPSHOT
[INFO]   from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- jacoco:0.8.12:prepare-agent (prepare-agent) @ minimarket ---
[INFO] argLine set to "-javaagent:C:\Users\rzuni\.m2\repository\org\jacoco\org.jacoco-agent-0.8.12.jar=destfile=C:\Users\rzuni\iCloudDrive\Raul\DUOC\DUOC - Desarrollo de Aplicaciones_\07 - Bimestre\02 - Desarrollo de servicios con JUnit\minimarket\target\jacoco.exec"
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ minimarket ---
[INFO] Copying 1 resource from src\main\resources to target\classes
[INFO] Copying 0 resource from src\main\resources to target\classes
[INFO]
```

```
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.minimarket.service.impl.UsuarioServiceImplTest
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer be necessary in Mockito 2.0.0. See https://mockito.org/2.0.0-migration.html for more details.
WARNING: A Java agent has been loaded dynamically (C:\Users\rzuni\.m2\repository\org\mockito\mockito-core-2.9.0.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.enabled=true
WARNING: Dynamic loading of agents will be disallowed by default in a future release
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.001 s -- in com.minimarket.service.impl.UsuarioServiceImplTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
[INFO]
```

Como resultado de la prueba solo a `UsuarioServiceTest`, nos entregó que esta corrió a 6 pruebas con cero fallos, cero errores y cero saltos.

También se puede realizar lo mismo para VentaServiceImplTest mediante la ejecución del siguiente comando: `./mvnw.cmd -Dtest=VentaServiceImplTest test`

Imagen de la ejecución del comando de prueba.

```
PS C:\Users\rzuni\iCloudDrive\Raul\DUOC\DUOC - Desarrollo de Aplicaciones_\07 - Bimestre\02 - Desarrollo Backend II - micro servicios con JUnit\minimarket> ./mvnw.cmd -Dtest=VentaServiceImplTest test
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.minimarket:minimarket >-----
[INFO] Building minimarket 0.0.1-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- jacoco:0.8.12:prepare-agent (prepare-agent) @ minimarket ---
[INFO] argLine set to "-javaagent:C:\Users\rzuni\.m2\repository\org\jacoco\org.jacoco.agent\0.8.12\jacoco-agent.jar -Djava.class.path=C:\Users\rzuni\iCloudDrive\Raul\DUOC\DUOC - Desarrollo de Aplicaciones_\07 - Bimestre\02 - Desarrollo Backend II - micro servicios con JUnit\minimarket\target\jacoco.exec"
[INFO]
```

```
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.minimarket.service.impl.VentaServiceImplTest
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases as described in Mockito's documentation: https://javadoc.io/doc/org.mockito/mockito-core/latest/org/mockito/Mockito.html#1.14
WARNING: A Java agent has been loaded dynamically (C:\Users\rzuni\.m2\repository\net\bytebuddy\byte-buddy-agent\1.12.1\byte-buddy-agent.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been altered
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.253 s -- in com.minimarket.service.impl.VentaServiceImplTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jacoco:0.8.12:report (report) @ minimarket ---
[INFO] Loading execution data file C:\Users\rzuni\iCloudDrive\Raul\DUOC\DUOC - Desarrollo de Aplicaciones_\07 - Bimestre\02 - Desarrollo Backend II - micro servicios con JUnit\minimarket\target\jacoco.exec
[INFO] Analyzed bundle 'minimarket' with 30 classes
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] -----
[INFO] Total time: 6.692 s
[INFO] Finished at: 2026-06-14T13:03:26-04:00
[INFO]
[INFO] -----
PS C:\Users\rzuni\iCloudDrive\Raul\DUOC\DUOC - Desarrollo de Aplicaciones_\07 - Bimestre\02 - Desarrollo Backend II - micro servicios con JUnit\minimarket>
```

Si ingresamos a la siguiente ruta:

`\target\site\jacoco\com.minimarket.service.impl\index.html`

Podemos ver que el resultado que nos entrega JaCoCo es el siguiente:

 [minimarket](#) >  `com.minimarket.service.impl`

com.minimarket.service.impl

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Mi
 DetalleVentaServiceImpl		9 %		n/a	
 ProductoServiceImpl		9 %		n/a	
 CarritoServiceImpl		9 %		n/a	
 InventarioServiceImpl		9 %		n/a	
 CategoriaServiceImpl		11 %		n/a	
 UsuarioServiceImpl		87 %		65 %	
 VentaServiceImpl		89 %		59 %	
 RolServiceImpl		37 %		n/a	
Total	177 of 481	63 %	28 of 74	62 %	

- UsuarioServiceImpl: 87% de cobertura en instrucciones.
- VentaServiceImpl: 89% de cobertura en instrucciones.

4. Reflexion tecnica

Las pruebas unitarias aportan valor porque validan la lógica más sensible del sistema antes de llegar a producción.

En el caso de la aplicación que se está realizando para una empresa de super mercado o minimarket, un error pequeño en stock o total puede generar reclamos, pérdida de confianza en los clientes e incluso pérdidas económicas para el negocio.

Las pruebas que realizamos en esta semana son importantes para ayudar a:

- Detectar errores de negocio en etapas tempranas.
- Confirmar que las reglas de validación no se rompen al modificar el código.
- Aislar dependencias con Mockito para obtener resultados deterministas.
- Medir cobertura con JaCoCo y encontrar huecos de validación.

Respuestas a las preguntas de apoyo

- ¿Qué estrategias podrías usar para simular dependencias entre ventas y usuarios en las pruebas unitarias?

- Para simular dependencias entre ventas y usuarios, una buena estrategia es usar mocks de repositorios y servicios, como `UsuarioRepository`, `VentaRepository` y `ProductoService`.

- ¿Cómo validarías que las ventas sin stock no se procesen?

- Para validar que las ventas sin stock no se procesen, se debe verificar el stock disponible antes de guardar la venta y lanzar una excepción o devolver falso si no alcanza.

- ¿Qué ventajas tiene medir la cobertura de código en un sistema con múltiples entidades relacionadas?

- Medir cobertura de código ayuda a entender que partes críticas del negocio están realmente ejercitadas por pruebas y evita confiar solo en ejecución manual.



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC. La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.